

Identity Column in SQL Server with Examples

In this article, I am going to discuss **Identity Column in SQL Server** with examples. Please read our previous article where we discussed [Cascading Referential Integrity Constraints in SQL Server](#) with examples. As part of this article, we are going to discuss the following pointers related to Identity Column.

1. **What is Identity in SQL Server?**
2. **Syntax and Example of Identity Column in SQL Server.**
3. **Explicitly supply Values for Identity Column in SQL Server?**
4. **How to Reset the Identity Column Value in SQL Server?**
5. **How we will get the last generated identity column value in SQL Server?**
6. **Differences between SCOPE_IDENTITY(), @@IDENTITY and IDENT_CURRENT('TableName') in SQL Server.**

What is Identity in SQL Server?

The Identity in SQL Server is a property that can be applied to a column of a table whose value is automatically created by the server. So, whenever you marked a column as identity, then that column will be filled in an auto-increment way by SQL Server. That means as a user we cannot insert a value manually into an identity column.

Syntax: **IDENTITY [(seed,increment)]**

Arguments:

1. **Seed:** Starting value of a column. The default value is 1.
2. **Increment:** It specifies the incremental value that is added to the identity column value of the previous row. The default value is 1.

We can set the identity property to a column either when the table is created or after table creation. The following shows an Identity property when the table is created:

```
Create Table Person
(
    PersonId int identity(1, 1),
    Name nvarchar(20)
)
```

The following example shows an Identity column after the table has been created:

```
CREATE TABLE Person
(
    PersonId int,
    Name nvarchar(20)
)
GO
ALTER TABLE Person
DROP COLUMN PersonId;
```

```
GO
ALTER TABLE Person
ADD PersonId INT IDENTITY(1,1);
GO
```

It is also possible to set the identity property of a column after the table is created. In such cases, first, we need to drop that column and then create that column using the identity property. If a column is marked as an identity column, then the values for this column are automatically generated, when we insert a new row into the table.

The above create table statement marks PersonId as an identity column with seed = 1 and Identity Increment = 1. Seed and Increment values are optional. If we don't specify the identity increment and seed, then by default both are to 1.

Identity Column Examples in SQL Server

Consider the following 2 insert statements, here we only pass the values for the Name column. We are not passing the value for the PersonId column.

```
Insert into Person values ('Bob')
Insert into Person values ('James')
```

But, If we select all the rows from the Person table, then we will see that, 'Bob' and 'James' rows have got 1 and 2 as PersonId.

```
select * from Person
```

Name	PersonId
Bob	1
James	2

Now, if I try to execute the following query,

```
Insert into Person values (1,'Mark')
```

It will give us the following error

An explicit value for the identity column in table 'Person' can only be specified when a column list is used and IDENTITY_INSERT is ON.

So if we mark a column as an Identity column, then we don't need to supply a value for that column explicitly. The value for the identity column is automatically calculated and provided by SQL Server. So, to insert a row into the Person table, just provide value for the Name column as shown below.

```
Insert into Person values ('Mark')
```

Now fetch the record from the Person table

Select * from Person It will give the following result set.

Name	PersonId
Bob	1
James	2
Mark	3

Delete the row, that we have just inserted i.e. the row with PersonId = 3, and insert another row as shown below.

Delete from Person where PersonId = 3
Insert into Person values('Smith')

Now fetch the record from the Person table as **Select * from Person** which will give the following result set.

Name	PersonId
Bob	1
James	2
Smith	4

You can see that the value for PersonId is 4. A record with PersonId = 3, does not exist, and you want to fill this gap. To do this, you should be able to explicitly supply the value for the identity column.

How to explicitly supply Values for Identity Column in SQL Server?

To explicitly supply a value for the identity column

1. First, turn on identity insert – **SET Identity_Insert Person ON**
2. Secondly, you need to specify the identity column name in the insert query as shown below

Insert into Person(PersonId, Name) values(3, 'Sara')

Now fetch the record from the Person table as **Select * from Person** which should give the following result.

Name	PersonId
Bob	1
James	2
Sara	3
Smith	4

Let's execute the insert query as **Insert into Person values('Pam')** which will give us the following error.

Explicit value must be specified for identity column in table 'Person' either when **IDENTITY_INSERT** is set to **ON** or when a replication user is inserting into a **NOT FOR REPLICATION** identity column.

As long as the **Identity_Insert** is turned on for a table, we need to explicitly provide the value for that column. If we don't provide the value, we get an error as shown in the above example.

So once we filled the gaps in the identity column, and if we wish the SQL server to calculate the value, turn off **Identity_Insert** as shown below.

SET Identity_Insert Person OFF

How to Reset the Identity Column Value in SQL Server?

If you have deleted all the rows in a table, and you want to reset the identity column value, then you need to use the **DBCC CHECKIDENT** command. This command will reset the identity column value.

Syntax: DBCC CHECKIDENT(TableName, RESEED, 0)

Example: SET Identity_Insert Person OFF

Delete from Person — Delete all the records from the Person table

DBCC CHECKIDENT(Person, RESEED, 0) — Use DBCC command to reset the identity column value

Once we execute the DBCC Command it will give us output as Checking identity information: current identity value '4'. DBCC execution completed. If DBCC printed some error messages, then just contact your database administrator.

```
-- Insert some data into the Person table
Insert into Person values ('Bob')
Insert into Person values ('James')
```

Select * from Person will give us the below result.

Name	PersonId
Bob	1
James	2

How we will get the last generated identity column value in SQL Server?

We already understood that the identity column values are auto-generated by SQL Server. In many scenarios, we want to get the last generated identity column value. In SQL Server, there are many different ways we can use to retrieve the last generated identity value. The most common approach is to use the built-in **SCOPE_IDENTITY()** function.

Apart, from using **SCOPE_IDENTITY()** built-in function, you may also use the built-in **@@IDENTITY** and **IDENT_CURRENT('TableName')** function.

Let us see some examples by using the above built-in function for getting the last generated identity column value in SQL Server.

```
Select SCOPE_IDENTITY()  
Select @@IDENTITY  
Select IDENT_CURRENT('Person')
```

Difference between COPE_IDENTITY(), @@IDENTITY and IDENT_CURRENT('Person') in SQL Server.

The **SCOPE_IDENTITY()** returns the last identity value that is created in the same session (Connection) and in the same scope (in the same Stored procedure, function, trigger). Let's say, I have 2 tables Person1 and Person2 and I have a trigger on the Person1 table, which will insert a record into the Person2 table.

Now, when we insert a record into the Person1 table, **SCOPE_IDENTITY()** returns the identity value that is generated in the Person1 table, whereas **@@IDENTITY** returns, the value that is generated in the Person2 table.

So **@@IDENTITY** returns the last identity value that is created in the same session without any consideration to the scope. The **IDENT_CURRENT('Person')** returns the last identity value created for a specific table across any session and any scope.

Note: We will discuss the example when we discuss Triggers in SQL Server.

In Short:

1. **SCOPE_IDENTITY():** The **SCOPE_IDENTITY()** built-in function returns the last identity column value that is created within the same session and the same scope.
2. **@@IDENTITY:** The **@@IDENTITY()** built-in function returns the last identity column value that is created in the same session but with any scope.
3. **IDENT_CURRENT('TableName'):** The **IDENT_CURRENT()** built-in function returns the last identity column value that is created for a specific table across any session and any scope.

In the next article, I am going to discuss **Sequence Object in SQL Server** which is a new feature introduced in SQL server 2012. Here, in this article, I try to explain the **Identity Column in SQL Server** with Examples. I hope you enjoy this Identity Column in SQL Server with Examples article.